UAlg – ISE – DEE
Algoritmos e Estruturas de Dados

TP1 — Ordenação de Inteiros

Laura Kayamori · N.º 85237
Universidade do Algarve · 26/10/2026

Resumo
Este trabalho apresenta uma aplicação em Java para ler, ordenar e gravar conjuntos de inteiros em ficheiros de texto e binários. Foram implementados Selection Sort, Bubble Sort e Insertion Sort, e comparados os seus tempos de execução sobre os mesmos dados. Os testes concluídos até 500.000 elementos evidenciam diferenças de desempenho entre algoritmos com complexidade quadrática.
Keywords — Java; ordenação; Selection Sort; Bubble Sort; Insertion Sort; ficheiros; desempenho

1. Introdução

O programa recebe pela linha de comandos o caminho de um ficheiro e a quantidade exata de inteiros a processar. Lê os dados, executa três algoritmos de ordenação sobre cópias independentes, verifica os resultados e guarda os valores ordenados em texto e binário. No Main, o Selection Sort foi escolhido para a escrita pela simplicidade da implementação; os ensaios mostram que não é necessariamente o mais rápido. Apesar de o Insertion Sort apresentar melhores tempos nos conjuntos maiores, manteve-se o Selection Sort na aplicação principal pela simplicidade da sua lógica, baseada na procura sucessiva do menor elemento. O objetivo foi privilegiar uma implementação fácil de compreender e verificar, enquanto a comparação de desempenho é realizada separadamente no Benchmark.

2. Algoritmos de ordenação estudados

Algoritmo	Lógica	Melhor caso	Médio/pior
Selection	Seleciona o menor e troca-o com a posição atual.	$O(n^2)$	$O(n^2)$
Bubble (básico)	Compara vizinhos e troca valores fora de ordem.	$O(n^2)$	$O(n^2)$
Insertion	Desloca valores maiores para inserir a chave.	$O(n)$	$O(n^2)$

Tabela 1 - Funcionamento e complexidade dos algoritmos de ordenação

Os três algoritmos ordenam os números diretamente no próprio array, utilizando apenas algumas variáveis adicionais. No Selection Sort e no Bubble Sort implementados, o número de comparações aumenta bastante quando o conjunto de dados cresce, mesmo que os números já estejam ordenados. O Insertion Sort funciona de maneira diferente, pois pode ser mais rápido quando os números já estão organizados ou quase organizados. Por isso, no seu melhor caso, apresenta complexidade $O(n)$, enquanto os outros dois apresentam $O(n^2)$. Esta notação representa a forma como o trabalho de cada algoritmo aumenta com a quantidade de números.

3. Implementação do trabalho

A aplicação está organizada em quatro classes. Main valida argumentos, escolhe a leitura do ficheiro, executa os algoritmos e apresenta os resultados, além de chamar o FileManager, que reúne os métodos de leitura e escrita. SortingAlgorithms contém os três algoritmos de ordenação.

Benchmark gera os dados e mede os tempos sem incluir operações de ficheiro. Para comparar os resultados, cada algoritmo recebe original.clone().
É também criada uma cópia esperado, ordenada com Arrays.sort(); Arrays.equals() verifica se o resultado de cada algoritmo corresponde à referência. A aplicação guarda os números produzidos pelo Selection Sort, após a validação.

3.1 Metodologia experimental

Os dados são gerados em memória com Random(42) e nextInt(100000), obtendo inteiros entre 0 e 99.999 de forma reproduzível. Para cada tamanho, são realizadas três execuções por algoritmo. System.nanoTime() é chamado imediatamente antes e depois da ordenação; a diferença é convertida para milissegundos e calcula-se a média. A preparação dos dados, cópia e validação ficam fora do intervalo cronometrado.
É realizado um aquecimento simples da JVM com 1.000 elementos por algoritmo. Esse aquecimento não elimina todas as variações decorrentes de JIT, recolha de lixo ou da atividade do sistema.

3.2 Resultados experimentais

Quantidade	Selection (ms)	Bubble (ms)	Insertion (ms)
1.000	0,545	1,651	3,266
5.000	11,278	19,885	4,918
10.000	43,085	69,971	13,103
100.000	4.490,059	20.231,927	2.611,553
250.000	30.590,604	146.516,139	16.622,645
500.000	156.531,768	613.429,160	74.267,645

Tabela 2 - Tempo de execução

Os testes foram realizados com conjuntos de 1.000 até 500.000 números inteiros, sendo cada algoritmo executado três vezes para calcular o tempo médio.
Nos conjuntos mais pequenos, os tempos foram reduzidos e apresentaram algumas variações. No teste de 1.000 inteiros, o Selection Sort obteve a menor média (0,545 ms), enquanto nos testes de 5.000 e 10.000 inteiros o Insertion Sort foi mais rápido.
A partir dos 100.000 inteiros, as diferenças tornaram-se mais evidentes. O Insertion Sort apresentou os menores tempos médios, seguido do Selection Sort. O Bubble Sort foi o mais lento, chegando aos 613.429,160 ms para 500.000 inteiros, aproximadamente 10 minutos por execução.
Ao comparar os conjuntos de 250.000 e 500.000 inteiros, verificou-se um aumento significativo dos tempos de execução, isto acontece porque estes algoritmos precisam de realizar muito mais comparações e deslocamentos à medida que a quantidade de números aumenta.
Também foram observadas diferenças entre as três repetições de alguns testes, possivelmente relacionadas com o funcionamento da JVM e com a atividade do computador durante as medições.
Apesar de os três algoritmos terem um crescimento semelhante no número de operações para conjuntos aleatórios, os tempos medidos foram diferentes. Assim, percebemos que dois algoritmos podem ter uma complexidade semelhante e, mesmo assim, apresentar desempenhos diferentes na prática.

4. Modo de funcionamento

O programa é executado através da linha de comandos, recebendo dois argumentos: o caminho do ficheiro e a quantidade exata de inteiros a processar. Por exemplo, java -cp out Main dados/numeros.txt 5.
A quantidade é convertida com Integer.parseInt() e são rejeitados valores inválidos ou não positivos. O programa identifica o formato através da extensão .bin, escolhendo o método de leitura correspondente.

4.1 Ficheiros de entrada e saída

São suportados dois formatos:
Texto: contém números inteiros representados por caracteres, normalmente um por linha. A leitura é realizada com Scanner e a escrita com PrintWriter. Este formato foi utilizado por ser legível e permitir verificar facilmente os valores.
Binário: cada inteiro ocupa quatro bytes, sendo utilizado DataInputStream.readInt() para leitura e DataOutputStream.writeInt() para escrita. Os valores são representados em formato big-endian. Este formato ocupa exatamente 4 × n bytes e permite guardar os inteiros diretamente na sua representação binária.
Os dois formatos foram implementados para cumprir os requisitos do trabalho e comparar diferentes formas de armazenamento. Independentemente do formato recebido, a aplicação guarda os resultados em resultados/ordenados.txt e resultados/ordenados.bin.

4.2 Validações

Na leitura de texto, o programa verifica se existem exatamente os inteiros indicados, rejeitando dados em falta, inválidos ou adicionais. Na leitura binária, verifica se o ficheiro possui o tamanho esperado. Depois de escrever o resultado binário, volta a lê-lo para confirmar que os números foram preservados.

5. Conclusão e trabalhos futuros

Com este trabalho foi possível implementar três algoritmos de ordenação em Java e comparar o seu funcionamento e desempenho. Também foram utilizados ficheiros de texto e binários, permitindo compreender melhor as diferenças entre estas duas formas de guardar números.
Durante os testes, verificou-se que o Insertion Sort foi o mais rápido nos conjuntos maiores, enquanto o Bubble Sort apresentou os tempos mais elevados. À medida que a quantidade de números aumentou, o tempo necessário para ordenar também aumentou significativamente.
O trabalho ajudou a compreender a importância de testar os algoritmos com diferentes quantidades de dados, verificar se os resultados estão corretos e comparar os tempos de execução antes de escolher um método de ordenação.
Como melhoria futura, poderia ser implementada uma versão do Bubble Sort que terminasse quando os números já estivessem ordenados. Também seria interessante estudar outros algoritmos mais rápidos para grandes quantidades de dados.

6. Condições de reprodução e limitações

Sistema: Windows 11; JDK: Eclipse Adoptium 25.0.4.1; processador: AMD Ryzen 7 5825U e memória RAM: 16GB. O relatório foi preparado a partir dos resultados de terminal registados.

Referências

Enunciado do 1.º Trabalho de Algoritmos e Estruturas de Dados, 2026/27 (fornecido pelo docente).
Documentação oficial Java SE: System.nanoTime(), Arrays.sort(), DataInputStream e DataOutputStream (APIs utilizadas).